LABORATORIO DE SISTEMAS

Tema 4 - Gestión de cuentas, usuarios y grupos

Felipe Ortega, Daniela Feversani, Elena García-Morato

Laboratorio de Sistemas (LSIS)
GSyC, EIF. URJC

2026

GSyC

Universidad Rey Juan Carlos

Licencia



(cc) 2025-2026 Felipe Ortega, Daniela Feversani, Elena García-Morato
Algunos derechos reservados. Este trabajo se entrega bajo la licencia Creative Commons Reconocimiento - NoComercial - SinObraDerivada (by-nc-nd). Para obtener la licencia completa, véase https://creativecommons.org/licenses/by-nc-nd/3.0/es/.

Contenidos

- 4.1 Usuarios y grupos
- 4.2 Gestión de usuarios
- 4.3 Gestión de grupos

4.1 Usuarios y grupos

La cuenta de usuario en Unix/Linux

- Los sistemas Unix/Linux son **multiusuario**: varios usuarios pueden usar el mismo sistema, incluso de forma simultánea (conexión remota o local).
- Ejemplo: los propios PCs de laboratorios de la universidad.
- En un sistema Unix/Linux tenemos **usuarios** y **grupos**.
- **Todo usuario pertenece a un grupo**. Se puede consultar esta información con los comandos `whoami` e `id`.

```
jfelipe@f-l3103-pc23:~$ whoami
jfelipe
jfelipe@f-l3103-pc23:~$ id
uid=6017(jfelipe) gid=600(profes) grupos=600(profes),20(dialout),29(audio),46(plugdev),108(kvm),...
```

La cuenta de usuario en Unix/Linux

- Los usuarios permiten identificar quién ha lanzado un proceso y determinar los permisos con los que ese proceso se ejecuta.
- El proceso de autenticar a un usuario para que acceda a su cuenta se denomina ***login***.
- En la terminal, este proceso lo realiza el programa `login`. En entornos gráficos, lo gestionan programas como `gdm` (GNOME) o `SDDM` (KDE).
- El usuario introduce su **nombre de usuario** y su **contraseña** para que el sistema lo autentique y le permita iniciar sesión.
- El **acceso remoto** suele implicar más pasos, como autenticación multifactor (por ejemplo, mediante el móvil) o el uso de certificados criptográficos.

Pseudo-usuarios

- Son cuentas del sistema que no pueden iniciar sesión (no hacen *login*).
- Se utilizan para ejecutar procesos y servicios del sistema (demonios).
- Permiten que cada servicio se ejecute con un usuario propio, limitando sus privilegios.
- **Ejemplo:**
  - El servidor web se ejecuta como `www-data`.
  - Si el servidor es comprometido, solo tendrá los permisos de `www-data`, no los de `root`.

El directorio $HOME

- Cada usuario tiene un directorio etiquetado como `$HOME`.
- Se traduce automáticamente a la ruta de su propia carpeta de usuario.
- Recordemos que el símbolo $\sim$ también se traduce a la ruta a la carpeta `$HOME` del usuario.

Identificador de usuario y de grupo

- En el *kernel* los usuarios y grupos se identifican mediante etiquetas numéricas.
  - **Identificador de usuario**: `uid`.
  - **Identificador de grupo**: `gid`.
- En el espacio de usuario trabajamos con nombres (*usernames* o *login names*).
- Los programas de espacio de usuario traducen entre los **números** (del kernel) y los **nombres** (espacio de usuario) usando **ficheros que mantienen datos de correspondencia entre ambos**.

Fichero /etc/passwd

- Fichero de texto con la traducción entre *login names* y `uids` de usuarios/as.
- Contiene más información.

```
paurea@alpha01:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
pulse:x:115:122:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
paurea:x:1000:1001:Gorka Guardiola:/home/paurea:/bin/bash
```

Fichero /etc/passwd

- Contiene una línea por cada usuario, en la cual los datos de cada campo están separados por el carácter :
- *Nombre de usuario*.
- *Contraseña* encriptada. (almacenada en /etc/shadow)
- *Identificador de usuario* (`uid`) y de *grupo* primario (`gid`).
- *Identificador del usuario en formato legible* (para humanos).
- *Ruta al directorio* `$HOME`.
- *Ruta al intérprete de línea de comandos* para ese usuario.

```
paurea:x:1000:1001:Gorka Guardiola:/home/paurea:/bin/bash
```

Fichero /etc/passwd

- Si el intérprete de comandos es `nologin` o se indica en ese campo el valor `false` entonces no se permite a ese usuario hacer *login* (para demonios).

```
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

Fichero /etc/passwd

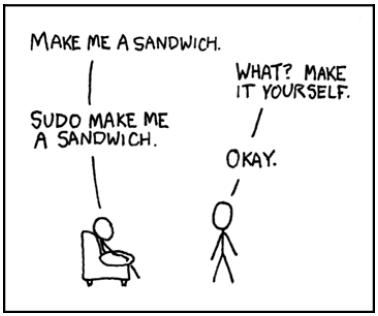
- Las contraseñas se almacenan en otro fichero, `/etc/shadow`, que no permite que todos lo lean (para evitar que cualquiera tenga acceso).
- Un *hash* seguro (resumen) de la contraseña de cada usuario.
- Los campos también están separados por el carácter `:`
- Contiene también fechas de cambio de contraseñas, tiempo máximo de validez, etc.
- Las cuentas desactivadas tienen como contraseña el símbolo `!`
- Más información de `shadow` en [2].

Superusuario `root`

- El usuario que tiene los máximos privilegios en el sistema es el superusuario `root`.
- Tiene identificadores de usuario y grupo especiales.
  - `UID = 0`.
  - `GID = 0`.
- Posee permisos totales para hacer cualquier acción en el sistema; **su uso indebido puede comprometer gravemente la seguridad y estabilidad del mismo**.
- Normalmente, muchos sistemas deshabilitan el *login* remoto del usuario `root` por razones de seguridad.

Superusuario `root`

- El comando **sudo** (*superuser do*) permite ejecutar un comando con privilegios elevados, normalmente como el usuario `root`.
- El comando **su** (*substitute user*) permite cambiar a otro usuario; comúnmente se utiliza para convertirse en el usuario `root`.



4.2 Gestión de usuarios

Creación y eliminación de usuarios

- Casi nunca editamos los ficheros con información de usuarios/grupos a mano. Es mejor usar herramientas que automatizan el proceso.
- El comando `adduser username` permite añadir un nuevo usuario.
- El comando `passwd username` permite cambiar la contraseña de un usuario.
- El comando deluser username permite borrar un usuario existente.

Credenciales y permisos usuarios

- Un **proceso** tiene *credenciales*, es decir, información sobre quién es el propietario del mismo (usuario y grupo).
- `PID`: Identificador de proceso.
- `UID`: Identificador de usuario propietario del proceso.
- `GID`: Identificador de grupo al que pertenece el usuario.
- `EUID`: Id. de usuario efectivo, que se usa realmente para comprobar permisos.
- `EGID`: Identificador de grupo efectivo, que realmente se emplea para comprobar permisos.

Nota: El sistema mira el EUID, no el UID, cuando verifica permisos.

Control y gestión de credenciales

- `/bin/id` **devuelve tu UID y GID.**
- `/bin/su` **ejecuta una shell con otro UID.** Por defecto, cambia al superusuario (UID = 0), pero se puede usar para cambiar a cualquier otro usuario.
- `/usr/bin/sudo` **ejecuta un comando con otro UID**, normalmente como root, pidiendo la contraseña del usuario actual. El fichero `/etc/sudoers` define quién puede convertirse en quién y qué comandos puede ejecutar.
- Ese fichero **solo se debe editar con el comando `visudo`**, nunca con otro editor.

Control y gestión de credenciales

- Ejemplo de entrada en el fichero `/etc/sudoers`:

```
pepe ALL = (root, bin : operator, system) ALL
```

- El usuario `pepe` puede, en cualquier máquina (`ALL`)
- ... adquirir el UID de `root` y `bin`
- ... adquirir el GID de `operator` y `system`
- ... para ejecutar cualquier comando.

Control y gestión de credenciales

- Otro ejemplo de entrada en `/etc/sudoers`:

```
paco laptop1 = NOPASSWD: /bin/kill, PASSWD: /bin/ls,
/usr/bin/lprm
```

- El usuario `paco` puede, en la máquina (`mono`)
- ... adquirir el UID de `root`
- ... para ejecutar `kill` sin meter contraseña
- ... para ejecutar `ls` o `lprm` metiendo contraseña.

Permisos POSIX

- Los ficheros y directorio tienen metadatos que informan sobre sus **permisos**.
  - Qué puede hacer el usuario (propietario) del fichero o directorio.
  - Qué pueden hacer el resto de usuarios que pertenecen al mismo grupo del propietario.
  - Qué pueden hacer el resto de usuarios del sistema (que no son el propietario ni pertenecen a su mismo grupo).

Permisos POSIX

- El comando `ls -l` muestra la información sobre ficheros, directorios y otras entradas en *formato largo*.
- `ls -ld` muestra sólo directorios.

```
-rw-rw-r--  1 jperez jperez     893 dic  1 20:10  file1
-rw-rw-r--  1 jperez jperez  198879 dic  1 20:21  file2
-rw-r--r--  1 jperez jperez    8980 may 13  2021  file3
drwxrwxr-x 15 jperez jperez    4096 feb 14 23:00  mydir
```

Permisos POSIX

- Para cada entrada del listado aparece:
  - Permisos: Los 10 primeros caracteres (incluido el primero, tipo de elemento).
  - Número de nombres del elemento (*hard links*).
  - Usuario propietario del elemento.
  - Grupo del propietario del elemento.
  - Tamaño en bytes.
  - Fecha y hora de la última modificación.
  - Nombre del elemento.

Permisos POSIX

```
-rw-r-r- 1 jperez jperez 8980 may 13 2021 file3
drwxrwxr-x 15 jperez jperez 4096 feb 14 23:00 mydir
```

- El primer carácter indica tipo de elemento.

`-`	Regular file - Fichero ordinario
`d`	Directory - Directorio
`l`	(Symbolic) Link - Enlace simbólico
`p`	Named pipe - Pipe con nombre
`s`	Socket - Socket
`c`	Character device - Dispositivo orientado a carácter
`b`	Block device - Dispositivo orientado a bloque

Permisos POSIX

- Los permisos de cada elemento (fichero, directorio, etc.) se codifican con los 9 últimos caracteres que aparecen al comienzo.

`-rwxrw-r--`

- El primer carácter es el tipo de elemento (ver diapositiva previa).
- Los caracteres del 2 al 4 indican los permisos del usuario propietario.
- Los caracteres del 5 al 7 indican los permisos del resto de usuarios del mismo grupo que el propietario.
- Los caracteres del 8 al 10 muestran los permisos del resto de usuarios del sistema (que no están en el mismo grupo que el propietario).

Permisos POSIX

- Al conjunto de permisos de un elemento (fichero, directorio, etc.) se le conoce como ***modo de acceso* (*access mode*)**.
- Cada grupo de tres caracteres indica si el permiso está concedido (aparece la letra correspondiente) o no lo está (aparece –).
- `r`: **permiso de lectura**. En un directorio permite listar las entradas que contiene.
- `w`: **permiso de escritura**. En un directorio permite modificar sus entradas: crear, borrar, renombrar o mover elementos.
- `x`: **permiso de ejecución**. En un directorio permite acceder a él o atravesarlo para llegar a archivos contenidos. Es necesario para leer datos o metadatos de los ficheros dentro del directorio.

Permisos POSIX

- Los permisos suelen representarse en **octal** al gestionarlos con comandos. Por ejemplo, **0664** corresponde a:

  `rw-` para el *propietario*,
  `rw-` para el *grupo* y
  `r-` para los *demás usuarios*.

Otros permisos especiales

- **Sticky bit (+t)**: se usa en directorios para impedir que un usuario borre o renombre archivos que no son suyos. Solo puede hacerlo el dueño del archivo, el dueño del directorio o `root`. Por ejemplo, se usa en `/tmp`.

```
drwxrwxrwt  21 root root  36864 feb 27 23:51 tmp
```

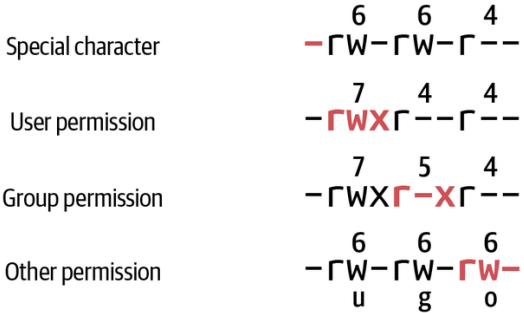
- **setuid/setgid bit (+s)**: en archivos ejecutables, hace que el proceso adopte el UID o GID del dueño o grupo del fichero al ejecutarlo. En Linux, normalmente se ignora en scripts interpretados (por ejemplo un script de bash, Python, etc.).

Permisos POSIX

- El comando `chmod` cambia los permisos de un fichero. Solo lo puede hacer el dueño del fichero y `root`.
- Inicialmente, el creador de un fichero es su dueño y grupo.
- El comando `chown` cambia el dueño de un fichero. Hay que tener privilegios especiales para hacer esto.
- El comando `chgrp` cambia el grupo de un fichero. El dueño puede cambiarlo a un grupo al que él pertenezca.
- **Cuidado**: En Linux los permisos POSIX conviven con otros permisos más potentes llamados ACLs (*Access Control Lists*). Véase `man 5 acl`.

Permisos con ***chmod*** (change mode)

chmod se puede utilizar con las designaciones **rwx** y **ugo**, pero **no se puede combinar ambas en el mismo comando**.



Cambio de permisos con `chmod` (change mode)

Dos formas de usar `chmod`:

- **1. En octal (números):** Permite establecer permisos completos de una sola vez.
  Algunos ejemplos comunes:
  - `chmod 0777 archivo` → asigna `rwx` a todos
  - `chmod 0755 archivo` → `rwx` al dueño, `rx` a grupo y otros
  - `chmod 0644 archivo` → `rw-` al dueño, `r-` a grupo y otros
  - `chmod 0600 archivo` → `rw-` solo al dueño, nadie más tiene permisos
  - `chmod 0700 archivo` → `rwx` solo al dueño, nadie más puede leer, escribir o ejecutar

Cambio de permisos con chmod (simbólico)

- **2. Con letras (simbólico):** Permite cambiar permisos específicos de manera precisa usando letras.
- **Designaciones de usuarios:**
  - u = usuario / dueño
  - g = grupo
  - o = otros
  - a = todos (usuario + grupo + otros)
- **Nota:** Si no se indica nada antes del signo (+, −, =), chmod asume a (all/todos).

Ejemplos de chmod con letras

- `chmod u-w archivo` → quita escritura solo al **usuario**
- `chmod g-w archivo` → quita escritura solo al **grupo**
- `chmod o-w archivo` → quita escritura solo a **otros**
- `chmod a-w archivo` → quita escritura a **todos**
- `chmod u+x archivo` → agrega ejecución al **usuario**
- `chmod g+x archivo` → agrega ejecución al **grupo**
- `chmod o+r archivo` → asigna lectura a **otros**
- `chmod u=rwx,g=rx,o=r archivo` → establece permisos exactos

Aclaración: En chmod, u siempre apunta al **propietario del archivo**, por eso "usuario" y "dueño" significan lo mismo aquí.

Aplicar chmod recursivamente

- Opción `-R`: aplica permisos a todos los archivos y subdirectorios de un directorio.

```
chmod -R g-rwx directorio
```

En este caso, quita todos los permisos al grupo en ese directorio y en todo su contenido.

- Esto es útil para directorios grandes donde quieres cambiar permisos de manera uniforme.

4.3 Gestión de grupos

Identificadores de grupo

- Permiten que varios usuarios compartan permisos de manera práctica.
- Todos los usuarios que pertenecen al mismo grupo tienen los mismos permisos asignados a ese grupo.
- Se identifican mediante un *group name* (nombre de grupo) y un `GID`.
- Están definidos en el fichero `/etc/group`.
- Cada línea del fichero describe un grupo y sus campos están separados por el carácter `:`

Fichero /etc/group

- Nombre del grupo.
- Contraseña del grupo (no se usa).
- Identificador numérico del grupo `gid`.
- Lista de nombres de usuarios, separados por comas, que son miembros del grupo (además de los que lo tienen como grupo primario en el fichero `/etc/passwd`).

```
paurea@alpha01:~$ cat /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
sudo:x:27:paurea
audio:x:29:pulse,paurea
```

Fichero `/etc/group`

- Para dar permisos a usuarios se pueden añadir a un grupo (por ejemplo para acceso a servicios, gestionar recursos de impresión, etc.).
- Se les puede añadir al grupo manualmente en este fichero, pero no es flexible.
- Método más potente y flexible: PAM (*Pluggable Authentication Modules*).
- Autenticación dinámica: es un servicio que permite al administrador del sistema decidir cómo autentican a los usuarios/as las diferentes aplicaciones.
- `man 5 pam`.
- Podemos buscar módulos con `man -k pam_` (ojo al subrayado al final).
- Ficheros de configuración en el directorio `/etc/pam.d`.

Fichero /etc/group

- El comando `adduser username groupname` permite añadir un usuario de un grupo.
- El comando `deluser username groupname` permite eliminar un usuario de un grupo.
- Para crear un nuevo grupo `addgroup groupname`.
- Para borrar un grupo `delgroup groupname`.

Para saber más

- El capítulo 2 de [2] trata sobre trabajar con permisos y cuentas privilegiadas en Linux.
- El capítulo 4 de [1] explica los sistemas de control de acceso y permisos en Linux, así como buenas prácticas para su gestión.
- El libro de referencia sobre administración en Unix/Linux [3] incluye mucha información adicional y excelentes explicaciones:
  - Capítulo 4: Control de acceso y privilegios de `root`.
  - Capítulo 7: Gestión de usuarios.
- El capítulo 7 de [4] también incluye apartados relevantes sobre gestión de usuarios y ficheros de configuración para gestión de usuarios y permisos.

Referencias

[1] M. Hausenblas. *Learning Modern Linux*. O'Reilly Media, abr. de 2022.

[2] Kenneth Hess. *Practical Linux System Administration: A Guide to Installation, Configuration, and Management*. Sebastopol, CA: O'Reilly Media, 2023.

[3] E. Nemeth et al. *Unix and Linux System Administration Handbook*. 4ª ed. Pearson, 2010.

[4] B. Ward. *How Linux Works: What Every Superuser Should Know*. 3ª ed. No Starch Press, abr. de 2021.